PHÂN TÍCH THIẾT KẾ THUẬT TOÁN

Chương 5

QUY HOẠCH ĐỘNG

(Dynamic Programming)

GV: Bùi Văn Thượng
Email: thuong.bui@ut.edu.vn

NỘI DUNG CHÍNH

- Chiến lược quy hoạch động
- Bài toán Dãy Fibonacci
- Bài toán nhân dãy các ma trận
- Phương pháp quy hoạch động
- Bài toán dãy con chung dài nhất

Chiến lược quy hoạch động

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Triết lý của Quy hoạch động

- Nếu "Chia để trị" chia bài toán thành các bài toán con độc lập, thì Quy hoạch động được dùng khi các bài toán con có sự chồng lấn.
- Triết lý cốt lõi là: "Không bao giờ giải cùng một bài toán hai lần". Thay vì tính toán lại, ta lưu trữ kết quả của các bài toán con đã giải vào một bảng (mảng hoặc ma trận) và tái sử dụng chúng khi cần thiết.

Chiến lược quy hoạch động

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Đặc điểm nhận diện bài toán Quy hoạch động

Để biết một bài toán có thể giải bằng Quy hoạch động hay không, ta dựa vào hai dấu hiệu:

- Cấu trúc con tối ưu: Lời giải tối ưu của bài toán lớn có thể được xây dựng từ lời giải tối ưu của các bài toán con.
- Bài toán có các bài toán con gối nhau (còn gọi là bài toán con chồng lấn - bài toán có công thức quy nạp giống trong toán học): Sử dụng kết quả của bài toán nhỏ để giải quyết bài toán lớn (bài toán chia để trị)

Chiến lược quy hoạch động

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Quy trình 4 bước thiết kế giải thuật

Để giải một bài toán theo chiến lược này, chúng ta cần tuân thủ các bước sau:

- Phân tích cấu trúc: Xác định các bài toán con và mối liên hệ giữa chúng.
- Xây dựng công thức truy hồi: Thiết lập công thức tính giá trị tối ưu của bài toán lớn thông qua các bài toán con.
- Tính toán dưới lên (Bottom-up): Tính kết quả của các bài toán con nhỏ nhất trước, sau đó lấp đầy bảng phương án cho đến bài toán gốc.
- Truy vết (Tracing): Dựa trên bảng phương án đã xây dựng để tìm ra lời giải chi tiết (nếu cần).

Chiến lược quy hoạch động

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Ví dụ minh họa: Bài toán tìm đường đi trên lưới

Bài toán: Bạn cần đi từ ô (0,0) đến ô (m,n) trên một lưới ô vuông. Tại mỗi bước chỉ được đi sang phải hoặc đi xuống dưới. Mỗi ô có một chi phí nhất định. Tìm đường đi có tổng chi phí nhỏ nhất.

				n		
		0	1	2	**3**	4
	0	►3	4	7	6	3
m	**1**	5	4	1	▼5	21
	2	7	8	12	23	6
	3	0	3	8	2	7
	4	1	9	4	5	3

Chiến lược quy hoạch động

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Ví dụ minh họa: Bài toán tìm đường đi trên lưới

Để tìm đường đi ngắn nhất từ (0,0) đến (1,3) trên lưới, chúng ta sẽ áp dụng đúng quy trình 4 bước của chiến lược Quy hoạch động.

1. Phân tích bài toán

- Điểm bắt đầu: (0,0) có giá trị là 3.
- Điểm kết thúc: (1,3) có giá trị là 5.
- Quy tắc di chuyển: Chỉ có thể đi sang phải hoặc xuống dưới.
- Mục tiêu: Tìm đường đi có tổng chi phí tại các ô đi qua là nhỏ nhất.

		n				
		0	1	2	3	4
	0	►3	4	7	6	3
m	1	5	4	1	▼5	21
	2	7	8	12	23	6
	3	0	3	8	2	7
	4	1	9	4	5	3

Chiến lược quy hoạch động

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Ví dụ minh họa: Bài toán tìm đường đi trên lưới

2. Xây dựng công thức truy hồi

Gọi L[i][j] là tổng chi phí nhỏ nhất để đi từ (0, 0) đến ô (i, j).

- Để đến được ô (i, j), ta chỉ có thể đến từ ô bên trái (i, j-1) hoặc ô phía trên (i-1, j).
- Công thức:

L[i][j] = Giá trị tại ô (i, j) + min(L[i][j-1], L[i-1][j]).

		n				
		0	1	2	**3**	4
	0	►3	4	7	6	3
m	**1**	5	4	1	▼5	21
	2	7	8	12	23	6
	3	0	3	8	2	7
	4	1	9	4	5	3

Chiến lược quy hoạch động

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Ví dụ minh họa: Bài toán tìm đường đi trên lưới

3. Tính toán bảng phương án (Bottom-up)

Ta chỉ cần tính toán các ô cần thiết dẫn đến mục tiêu (1,3).

Bước 1: Tính hàng đầu tiên (chỉ có thể đi từ bên trái sang)

- L[0][0] = 3
- L[0][1] = L[0][0] + 4 = 7
- L[0][2] = L[0][1] + 7 = 14
- L[0][3] = L[0][2] + 6 = 20

		n				
		0	1	2	3	4
	0	►3 3	4 7	7 14	6 20	3
m	1	5	4	1	▼5	21
	2	7	8	12	23	6
	3	0	3	8	2	7
	4	1	9	4	5	3

Chiến lược quy hoạch động

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Ví dụ minh họa: Bài toán tìm đường đi trên lưới

3. Tính toán bảng phương án (Bottom-up)

Bước 2: Tính hàng thứ hai (ô có chỉ số m = 1)

- Ô (1,0): Chỉ có thể đi từ trên xuống.

  L[1][0] = L[0][0] + 5 = 3 + 5 = 8.

		n				
		0	1	2	3	4
	0	►3 3	4 7	7 14	6 20	3
m	1	5 8	4	1	▼5	21
	2	7	8	12	23	6
	3	0	3	8	2	7
	4	1	9	4	5	3

Chiến lược quy hoạch động

Ví dụ minh họa: Bài toán tìm đường đi trên lưới

3. Tính toán bảng phương án (Bottom-up)

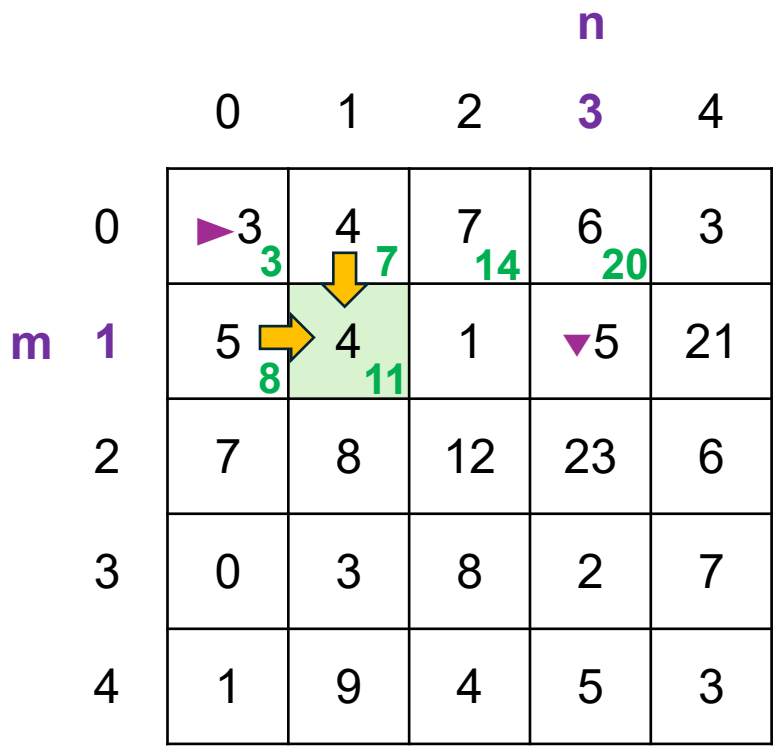
Bước 2: Tính hàng thứ hai (ô có chỉ số m = 1)

- Ô (1,1): Có thể từ (1,0) sang hoặc (0,1) xuống.

L[1][1] = 4 + min(L[1][0], L[0][1])

= 4 + min(8, 7)

= 4 + 7 = 11.



Chiến lược quy hoạch động

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Ví dụ minh họa: Bài toán tìm đường đi trên lưới

3. Tính toán bảng phương án (Bottom-up)

Bước 2: Tính hàng thứ hai (ô có chỉ số m = 1)

- Ô (1,2): Có thể từ (1,1) sang hoặc (0,2) xuống.

$$L[1][2] = 1 + \min(L[1][1], L[0][2])$$
$$= 1 + \min(11, 14)$$
$$= 1 + 11 = 12.$$

m \ n	0	1	2	3	4
0	►3 (3)	4 (7)	7 (14)	6 (20)	3
1	5 (8)	4 (11)	1 (12)	▼5	21
2	7	8	12	23	6
3	0	3	8	2	7
4	1	9	4	5	3

Chiến lược quy hoạch động

Ví dụ minh họa: Bài toán tìm đường đi trên lưới

3. Tính toán bảng phương án (Bottom-up)

Bước 2: Tính hàng thứ hai (ô có chỉ số m = 1)

- Ô (1,3) (Đích):

  Có thể từ (1,2) sang hoặc (0,3) xuống.

  L[1][3] = 5 + min(L[1][2], L[0][3])

  = 5 + min(12, 20)

  = 5 + 12 = 17.

m \ n	0	1	2	3	4
0	3 (3)	4 (7)	7 (14)	6 (20)	3
1	5 (8)	4 (11)	1 (12)	5 (17)	21
2	7	8	12	23	6
3	0	3	8	2	7
4	1	9	4	5	3

Chiến lược quy hoạch động

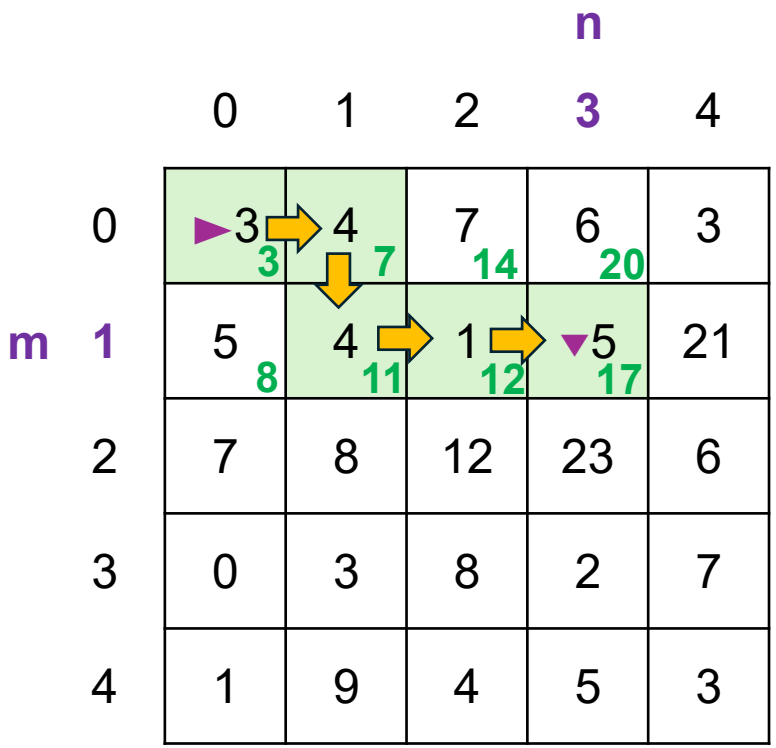
UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Ví dụ minh họa: Bài toán tìm đường đi trên lưới

4. Kết luận và Truy vết

- Tổng chi phí nhỏ nhất (Min Cost): 17.
- Đường đi cụ thể (Truy vết ngược):

- Từ ô (1,3) [trị giá 5], ta chọn ô có chi phí tích lũy nhỏ hơn giữa bên trái (12) và bên trên (20) → Chọn ô (1,2).
- Từ ô (1,2) [trị giá 1], chọn giữa bên trái (11) và bên trên (14) → Chọn ô (1,1).
- Từ ô (1,1) [trị giá 4], chọn giữa bên trái (8) và bên trên (7) → Chọn ô (0,1).
- Từ ô (0,1) quay về điểm bắt đầu (0,0).



Bài toán dãy Fibonacci

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Định nghĩa bài toán

Dãy Fibonacci được định nghĩa bởi công thức truy hồi:

- $F_0 = 0, F_1 = 1$
- $F_n = F_{n-1} + F_{n-2}$ với $n \geq 2$

Dãy số sẽ là: 0, 1, 1, 2, 3, 5, 8, 13, 21, ...

Bài toán dãy Fibonacci

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Vấn đề của cách tiếp cận Đệ quy thuần túy

- Sự chồng lấn (Overlapping):

  Để tính F_5, ta tính F_4 và F_3.

  Nhưng để tính F_4, ta lại tính F_3 và F_2.

  → Như vậy F_3 bị tính lại nhiều lần.

- Độ phức tạp: Thời gian chạy là hàm mũ $O(2^n)$.

  Với n=50, máy tính thông thường có thể mất hàng phút, thậm chí hàng giờ để xong.

```
int fibo(int n) {
    if (n <= 1) return n;
    return fibo(n-1) + fibo(n-2);
}
```

Bài toán dãy Fibonacci

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Giải quyết bằng Quy hoạch động (Dynamic Programming)

Cách 1: Phương pháp dưới lên (Bottom-up) - Dùng bảng

Ta dùng một mảng để lưu trữ kết quả từ nhỏ đến lớn.

- Độ phức tạp thời gian: O(n) – Chỉ cần một vòng lặp duy nhất.
- Độ phức tạp không gian: O(n) – Để lưu mảng

```
int fibDP(int n) {
    int f[n + 1];
    f[0] = 0;
    f[1] = 1;
    for (int i = 2; i <= n; i++) {
        f[i] = f[i-1] + f[i-2]; // Sử dụng kết quả đã lưu trong mảng
    }
    return f[n];
}
```

Bài toán dãy Fibonacci

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Giải quyết bằng Quy hoạch động (Dynamic Programming)

Cách 2: Tối ưu không gian (Space Optimized)

Nhận thấy để tính F_n, ta chỉ cần F_{n-1} và F_{n-2} →Ta có thể dùng 2 biến thay vì cả mảng.

Độ phức tạp không gian: O(1).

```
int fibOptimized(int n) {
    if (n <= 1) return n;
    int a = 0, b = 1, c;
    for (int i = 2; i <= n; i++) {
        c = a + b;
        a = b;
        b = c;
    }
    return b;
}
```

Bài toán dãy Fibonacci

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

So sánh hiệu năng thực tế

Giả sử tính F_{40}:

- Đệ quy: Thực hiện khoảng 331,160,281 lời gọi hàm.
- Quy hoạch động: Thực hiện đúng 40 phép cộng.

Bài toán nhân dãy các ma trận

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Đặt vấn đề

Giả sử ta cần nhân ba ma trận A, B, C. Phép nhân ma trận có tính chất kết hợp:

$$(A \times B) \times C = A \times (B \times C).$$

Tuy nhiên, số lượng phép nhân số thực (chi phí tính toán) ở hai cách này thường rất khác nhau.

Bài toán nhân dãy các ma trận

Ví dụ: Ma trận A kích thước 10 × 100, B (100 × 5), C (5 × 50)

Cách 1: (A × B) × C

- Nhân A × B tốn:

  10 × 100 × 5 = 5,000 phép nhân.

  → Kết quả là ma trận AB (10 × 5).
- Nhân (AB) × C tốn:

  10 × 5 × 50 = 2,500 phép nhân.
- Tổng cộng: **7,500** phép nhân.

Cách 2: A ×(B × C)

- Nhân B × C tốn:

  100 × 5 × 50 = 25,000 phép nhân.

  → Kết quả là ma trận BC (100 × 50).
- Nhân A ×(BC) tốn:

  10 × 100 × 50 = 50,000 phép nhân.
- Tổng cộng: **75,000** phép nhân.

Kết luận: Cách 1 nhanh gấp **10 lần** cách 2!

➔ Bài toán đặt ra là: Cho một dãy n ma trận, hãy tìm cách đặt dấu ngoặc sao cho tổng số phép nhân là ít nhất.

Bài toán nhân dãy các ma trận

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Phân tích theo Quy hoạch động

Bước 1: Cấu trúc con tối ưu

Để tính tích của dãy ma trận từ A_i đến A_j (ký hiệu $A_{i..j}$), ta phải chia dãy này tại một vị trí k nào đó ($i \leq k < j$):

$$A_{i..j} = (A_{i..k}) \times (A_{k+1..j})$$

Chi phí để tính $A_{i..j}$ bằng chi phí tính $A_{i..k}$ + chi phí tính $A_{k+1..j}$ + chi phí nhân hai ma trận kết quả cuối cùng.

Bài toán nhân dãy các ma trận

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Phân tích theo Quy hoạch động

Bước 2: Công thức truy hồi

Gọi m[i, j] là số phép nhân tối thiểu để tính tích $A_i \dots A_j$.

Giả sử ma trận A_i có kích thước $p_{i-1} \times p_i$.

- Nếu i = j: m[i, i] = 0 (một ma trận không cần nhân).
- Nếu i < j:

$$m[i, j] = \min\{ m[i, k] + m[k+1, j] + p_{i-1}.p_k.p_j\}$$

$$\text{Với } i \leq k < j$$

Bài toán nhân dãy các ma trận

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Thuật toán Quy hoạch động (Bottom-up)

Thay vì dùng đệ quy (tốn $O(2^n)$), ta dùng một bảng hai chiều m[n][n] để lưu kết quả. Ta tính toán theo độ dài của chuỗi ma trận: đầu tiên là các chuỗi độ dài 2, sau đó là 3, ..., cho đến n.

Bài toán nhân dãy các ma trận

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

```
void matrixChainOrder(int p[], int n) {
    int m[MAX][MAX];
    // n là số ma trận, mảng p có n+1 phần tử
    for (int i = 1; i <= n; i++) m[i][i] = 0;
    for (int len = 2; len <= n; len++) { // len là độ dài chuỗi ma trận
        for (int i = 1; i <= n - len + 1; i++) {
            int j = i + len - 1;
            m[i][j] = INF; // Khởi tạo vô cùng
            for (int k = i; k <= j - 1; k++) {
                int q = m[i][k] + m[k + 1][j] + p[i - 1] * p[k] * p[j];
                if (q < m[i][j]) {
                    m[i][j] = q;
                    // Có thể lưu lại k vào bảng s[i][j] để truy vết cách đặt ngoặc
                }
            }
        }
    }
    cout << "So phep nhan toi thieu: " << m[1][n];
}
```

Bài toán nhân dãy các ma trận

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Đánh giá thuật toán

- Độ phức tạp thời gian: $O(n^3)$ do có 3 vòng lặp lồng nhau (độ dài chuỗi, điểm bắt đầu, và điểm chia k).
- Độ phức tạp không gian: $O(n^2)$ để lưu trữ bảng m.

Tại sao đây là Quy hoạch động?

Bài toán này thỏa mãn cả hai điều kiện:

- Gối nhau (Overlapping subproblems): Khi tính m[1, 4], ta cần m[1, 2] và m[2, 4]. Khi tính m[1, 5], ta cũng lại cần m[1, 2] và m[2, 4].
- Cấu trúc con tối ưu: Lời giải cho dãy n ma trận được xây dựng trực tiếp từ lời giải tối ưu của các dãy ngắn hơn.

Phương pháp quy hoạch động

Trong khoa học máy tính, có hai cách tiếp cận chính để cài đặt một thuật toán Quy hoạch động: Top-down (Từ trên xuống) và Bottom-up (Từ dưới lên).

Cách tiếp cận Top-down (Memoization - Ghi nhớ)

Đây là cách tiếp cận giữ nguyên bản chất đệ quy của bài toán nhưng thêm một bước "lưu trữ".

- Cách làm: Ta giải bài toán bằng cách gọi đệ quy các bài toán con. Tuy nhiên, trước khi tính toán một bài toán con, ta kiểm tra xem nó đã được giải và lưu trong "sổ tay" (bảng băm hoặc mảng) chưa.
- Đặc điểm: Chỉ giải những bài toán con thực sự cần thiết.
  - Dễ cài đặt nếu bạn đã có sẵn công thức đệ quy.
  - Nhược điểm: Tốn bộ nhớ Stack cho đệ quy và có thể bị lỗi Stack Overflow nếu đệ quy quá sâu.

Phương pháp quy hoạch động

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Cách tiếp cận Bottom-up (Tabulation - Lập bảng)

- Cách làm: Ta bắt đầu giải từ các bài toán con nhỏ nhất (trường hợp cơ sở), sau đó sử dụng kết quả đó để xây dựng dần lên các bài toán lớn hơn cho đến khi đạt được bài toán gốc.
- Đặc điểm: Sử dụng vòng lặp (thường là for), không dùng đệ quy.
  - Tính toán tất cả các bài toán con trong bảng.
  - Ưu điểm: Tiết kiệm bộ nhớ Stack, tốc độ thường nhanh hơn Top-down do không mất chi phí gọi hàm.

Phương pháp quy hoạch động

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Sự khác biệt giữa Quy hoạch động và Chia để trị

Tiêu chí	Chia để trị (Divide and Conquer)	Quy hoạch động (Dynamic Programming)
Bản chất bài toán con	Độc lập (không trùng nhau).	Chồng lấn (được gọi lại nhiều lần).
Cách tính	Tính lại bài toán con mỗi khi gặp.	Tính một lần và lưu lại để dùng lại.
Ví dụ	Merge Sort, Quick Sort.	Fibonacci, Xâu con chung dài nhất.

Phương pháp quy hoạch động

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Quy trình tổng quát để giải bài toán quy hoạch động

- Định nghĩa trạng thái (State): Xác định ý nghĩa của các ô trong bảng phương án (ví dụ: L[i][j] là gì?).
- Thiết lập công thức truy hồi (State Transition): Tìm mối liên hệ giữa bài toán hiện tại và các bài toán con đã giải trước đó.
- Xác định điều kiện cơ sở (Base Case): Những giá trị nhỏ nhất mà ta biết ngay kết quả (thường là hàng 0, cột 0).
- Tính toán và Truy vết (Computation & Tracing): Điền bảng và đi ngược bảng để tìm lời giải chi tiết.

Bài toán dãy con chung dài nhất

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Bài toán dãy con chung dài nhất (Longest Common Subsequence - **LCS**) - Đây là bài toán nền tảng cho các công cụ so sánh tệp tin (như lệnh diff trong Linux), kiểm tra đạo văn, hoặc so sánh các chuỗi gene trong sinh học phân tử.

Định nghĩa bài toán

Cho hai dãy ký tự X và Y. Một dãy Z được gọi là dãy con chung của X và Y nếu Z có thể thu được bằng cách xóa đi một số ký tự trong X và Y (nhưng vẫn giữ nguyên thứ tự các ký tự còn lại).

Mục tiêu: Tìm dãy con chung có độ dài lớn nhất.

Bài toán dãy con chung dài nhất

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Ví dụ:

- X: ABCBDAB
- Y: BDCABA
- Dãy con chung có thể là BCB, BCBA, BDAB...
- Dãy con chung lớn nhất là BDAB hoặc BCBA (độ dài là 4).

Bài toán dãy con chung dài nhất

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Tại sao dùng Quy hoạch động?

Nếu thử mọi dãy con (Vét cạn), với dãy độ dài n, ta có 2^n dãy con. So sánh hai dãy sẽ tốn thời gian hàm mũ, không thể thực hiện được với các chuỗi dài.

Tuy nhiên, bài toán LCS có:

- Cấu trúc con tối ưu: LCS của hai chuỗi được xây dựng từ LCS của các chuỗi tiền tố ngắn hơn.
- Bài toán con chồng lấn: Khi so sánh X[1..i] và Y[1..j], ta lặp lại việc so sánh các đoạn mã trước đó nhiều lần.

Bài toán dãy con chung dài nhất

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Công thức truy hồi

Gọi L[i][j] là độ dài của dãy con chung dài nhất của i ký tự đầu tiên của X và j ký tự đầu tiên của Y.

- Trường hợp cơ sở: Nếu i=0 hoặc j=0, thì L[i][j] = 0.
- Nếu ký tự cuối giống nhau (X[i] == Y[j]):

  L[i][j] = L[i-1][j-1] + 1 (Ta tìm thấy 1 ký tự chung, cộng thêm vào kết quả của các phần trước đó).
- Nếu ký tự cuối khác nhau (X[i] ≠Y[j]):

  L[i][j] = max(L[i-1][j], L[i][j-1]) (Ta lấy kết quả tốt nhất khi bỏ đi 1 ký tự của X hoặc 1 ký tự của Y).

Bài toán dãy con chung dài nhất

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Thuật toán và Bảng phương án (Bottom-up)

Ta sử dụng một bảng hai chiều L[m+1][n+1] để lưu trữ các giá trị.

```
const int MAX = 100;
void findLCS(string X, string Y) {
    int m = X.length();
    int n = Y.length();
    int L[MAX][MAX];
    // 1. Xây dựng bảng phương án L[i][j]
    for (int i = 0; i <= m; i++) {
        for (int j = 0; j <= n; j++) {
            if (i == 0 || j == 0) L[i][j] = 0;
            else if (X[i - 1] == Y[j - 1])
                L[i][j] = L[i - 1][j - 1] + 1;
            else L[i][j] = max(L[i - 1][j], L[i][j - 1]);
        }
    }
    cout << "Do dai LCS: " << L[m][n] << endl;
    // 2. Truy vết để in ra dãy con chung
```

Bài toán dãy con chung dài nhất

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Thuật toán và Bảng phương án (Bottom-up)

```
void findLCS(string X, string Y) {
    //…
    //2. Truy vết để in ra dãy con chung
    string lcs_str = "";
    int i = m, j = n;
    while (i > 0 && j > 0) {
        if (X[i - 1] == Y[j - 1]) {
            lcs_str = X[i - 1] + lcs_str;
            i--; j--;
        } else if (L[i - 1][j] > L[i][j - 1])
            i--;
        else
            j--;
    }
    cout << "Day con chung dai nhat: " << lcs_str << endl;
}
```

Bài toán dãy con chung dài nhất

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Phân tích độ phức tạp

- Thời gian: O(m×n) với m, n là độ dài hai chuỗi. Đây là một bước tiến cực lớn so với $O(2^n)$.
- Không gian: O(m×n) để lưu bảng phương án.

Bài toán dãy con chung dài nhất

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Ý nghĩa thực tế

Trong học phần Phân tích thiết kế giải thuật, bài toán LCS dạy cho sinh viên cách:

- Sử dụng bảng 2 chiều để giải quyết các bài toán trên chuỗi.
- Cách truy vết (traceback) trên ma trận để lấy lại lời giải cụ thể thay vì chỉ lấy giá trị tối ưu.
- Tối ưu hóa tài nguyên:

  Nếu m, n rất lớn, ta có thể tối ưu bộ nhớ xuống O(min(m, n))).

BÀI TẬP CHƯƠNG 5

Bài 1. Hãy viết mã C++ để giải bài toán trong ví dụ tìm đường đi trên lưới vuông ở phần đầu của chương học.

Bài 2. Bài toán cái túi (Knapsack 0/1)

Một kẻ trộm có một cái túi có sức chứa trọng lượng tối đa là W. Có n đồ vật, mỗi đồ vật i có trọng lượng w_i và giá trị v_i. Kẻ trộm cần chọn các đồ vật sao cho tổng trọng lượng không quá W và tổng giá trị thu được là lớn nhất.

Yêu cầu:

A. Xác định trạng thái L[i][j] của bài toán.

B. Thiết lập công thức truy hồi.

C. Áp dụng cho dữ liệu: W = 5, đồ vật 1 (w=2, v=3), đồ vật 2 (w=3, v=4), đồ vật 3 (w=4, v=5).

BÀI TẬP CHƯƠNG 5

Bài 3. Dãy con tăng dài nhất (Longest Increasing Subsequence - LIS)

Cho một dãy số nguyên A = {a_1, a_2, ..., a_n}. Hãy tìm độ dài của dãy con tăng dài nhất trong A (các phần tử không cần liên tiếp nhưng phải giữ nguyên thứ tự).

- Ví dụ: A = {10, 22, 9, 33, 21, 50, 41, 60}. LIS là {10, 22, 33, 50, 60} có độ dài là 5.
- Yêu cầu: Thiết kế giải thuật độ phức tạp $O(n^2)$ bằng Quy hoạch động.

Bài 4. Bài toán cắt thanh kim loại (Rod Cutting)

Bạn có một thanh kim loại dài n inch và một bảng giá bán các đoạn có độ dài từ 1 đến n. Hãy tìm cách cắt thanh kim loại thành các đoạn nhỏ sao cho tổng giá trị bán được là lớn nhất.

Yêu cầu:

A. So sánh cách giải bằng Đệ quy (Top-down) và Lập bảng (Bottom-up).

B. Giải thích tại sao bài toán này có "Cấu trúc con tối ưu".

BÀI TẬP CHƯƠNG 5

Bài 5. Biến đổi chuỗi (Edit Distance / Levenshtein Distance)

Cho hai chuỗi ký tự S_1 và S_2. Các phép biến đổi cho phép gồm: Chèn, Xóa, Thay thế một ký tự. Tìm số phép biến đổi ít nhất để biến S_1 thành S_2.

Yêu cầu:

A. Xây dựng bảng phương án D[m][n].

B. Đây là bài toán mở rộng của LCS, hãy chỉ ra điểm giống và khác nhau trong công thức truy hồi của hai bài toán này.

BÀI TẬP CHƯƠNG 5

Bài 6. Tổng tập con (Subset Sum Problem)

Cho một tập hợp các số nguyên dương và một giá trị S. Kiểm tra xem có tồn tại một tập con nào có tổng các phần tử đúng bằng S hay không.

Yêu cầu:

A. Xây dựng bảng phương án kiểu Boolean.

B. Nếu các phần tử trong tập hợp có giá trị rất lớn, giải thuật Quy hoạch động này còn hiệu quả không? Tại sao? (Liên hệ với độ phức tạp giả đa thức - Pseudo-polynomial time).

THANKS